

MisterFix — Relatório de Integração

Disciplina	Lab. Desenvolvimento Aplicações Móveis e Distribuídas
Sprint	Sprint 2 — Integração com MOM
Entrega	25/05/2026
Tecnologias	Node.js + Express + RabbitMQ (amqplib) + SQLite

1. Ferramenta MOM Escolhida: RabbitMQ

O RabbitMQ foi escolhido como middleware orientado a mensagens por três razões principais:

- **Maturidade e adoção:** é o MOM open-source mais utilizado no mercado, com documentação extensa e suporte ativo.
- **Protocolo AMQP:** oferece garantias de entrega (at-least-once), filas duráveis e persistência de mensagens em disco — essencial para não perder notificações quando o app do prestador está offline.
- **Docker-ready:** a imagem oficial *rabbitmq:3-management* sobe em um comando, sem configuração adicional para desenvolvimento.

Para subir o RabbitMQ localmente: `docker run -d --name mrfix-rabbit -p 5672:5672 -p 15672:15672 rabbitmq:3-management`

Painel de administração disponível em `http://localhost:15672 (guest/guest)`.

2. Padrão de Integração Utilizado

O sistema utiliza o padrão Work Queue, também chamado de Task Queue. Neste padrão:

- O **produtor** (backend REST) publica mensagens em uma fila nomeada após salvar no banco de dados.
- O **consumidor** (mesmo processo backend, simulando o serviço de notificação) escuta a fila e processa as mensagens de forma assíncrona.
- O RabbitMQ atua como buffer — se o consumidor estiver sobrecarregado, as mensagens aguardam na fila sem bloquear a API REST.

As filas são declaradas como “durable: true” e as mensagens como “persistent: true”, garantindo que o RabbitMQ grave as mensagens em disco e não as perca em caso de reinicialização.

3. Documentação dos Eventos

A tabela 1 a seguir, demonstra os eventos criados, bem como os consumidores do evento e qual o gatilho de disparo.

Tabela 1 – Eventos desenvolvidos na Sprint 2

Evento	Produtor	Consumidor	Gatilho
service_request .created	serviceRequest Controller	requestCreated Consumidor	Cliente cria uma solicitação
service_request .accepted	serviceRequest Controller	requestAccepted Consumidor	Prestador aceita uma solicitação
service_request .status_changed	serviceRequest Controller	statusChanged Consumidor	Qualquer mudança de status
service_request .completed	serviceRequest Controller	statusChanged Consumidor	Prestador conclui o serviço

4. Decisões Tomadas de Design

Resiliência da API REST: a publicação de eventos é feita de forma não-bloqueante. Se o RabbitMQ estiver indisponível, a API REST continua operando normalmente — o evento é logado como perdido, mas a solicitação é salva no banco. Em produção, este comportamento seria substituído pelo padrão Outbox Table para garantia de entrega.

Prefetch = 1: cada consumidor processa uma mensagem por vez antes de receber a próxima. Isso garante distribuição justa de carga em cenários com múltiplas instâncias do consumidor rodando em paralelo.

Consumidores no mesmo processo: nesta sprint, produtores e consumidores rodam no mesmo processo Node.js para simplificar o desenvolvimento e a demonstração. Em produção, os consumidores seriam extraídos para um microserviço de notificações independente.

Reconexão automática: o gerenciador de conexão (config/rabbitmq.js) implementa reconexão com backoff fixo de 5 segundos. Isso permite que o servidor suba mesmo sem o RabbitMQ disponível e conecte assim que ele estiver pronto.

5. Desafios Encontrados na Aplicação

- **Race condition no startup:** o servidor tentava iniciar os consumidores antes da conexão com o RabbitMQ estar estável. Resolvido encapsulando o startup do MOM em try/catch independente do startup do banco.

- **Múltiplos eventos por transição:** uma mudança de status 'accepted' precisa publicar dois eventos (accepted + status_changed). Resolvido com publicações sequenciais no controller, pós-save.
- **Graceful shutdown:** sem tratamento adequado, o processo Node.js encerraria com a conexão AMQP aberta, causando erro no RabbitMQ. Resolvido com handlers SIGTERM/SIGINT que fecham canal e conexão antes de encerrar.

6. Evidências da Aplicação

Para validar a integração do middleware orientado a mensagens, foram realizados testes manuais cobrindo os principais eventos do domínio. As evidências coletadas demonstram o funcionamento assíncrono end-to-end entre o backend REST e o RabbitMQ.

Inicialização do Sistema: Ao iniciar o servidor, o log do terminal confirma que a conexão com o RabbitMQ foi estabelecida com sucesso e que os três consumidores foram registrados e estão ativos, aguardando mensagens nas respectivas filas: service_request.created, service_request.accepted e service_request.status_changed. O painel de administração do RabbitMQ, acessível em <http://localhost:15672>, exibe as quatro filas criadas automaticamente pelo sistema no momento da conexão, todas com o atributo durable ativo, garantindo persistência das mensagens em disco. Estas podem ser observadas nas figuras 1, 2 e 3, abaixo:

Figura 1 - Inicialização do servidor no Terminal

```
PS C:\Users\manti\OneDrive\Área de Trabalho\Projetos PUC\5 período\LABMOBILE\mrfix\backend> npm run dev
Sample request seeded

Seed complete!

Test credentials (password: senha123):
Client (female): ana@mrfix.com
Client (male): carlos@mrfix.com
Provider (female): julia@mrfix.com
Provider (male): marcos@mrfix.com
[RabbitMQ] Conectando em amqp://guest:guest@127.0.0.1:5672
[RabbitMQ] ✅ Conexão estabelecida.

[Messaging] Iniciando consumers...
[Consumer] 🟡 Escutando fila: service_request.created
[Consumer] 🟡 Escutando fila: service_request.accepted
[Consumer] 🟡 Escutando fila: service_request.status_changed
[Messaging] ✅ Todos os consumers ativos.
```

Figura 2 – Painel do RabbitMQ no localhost

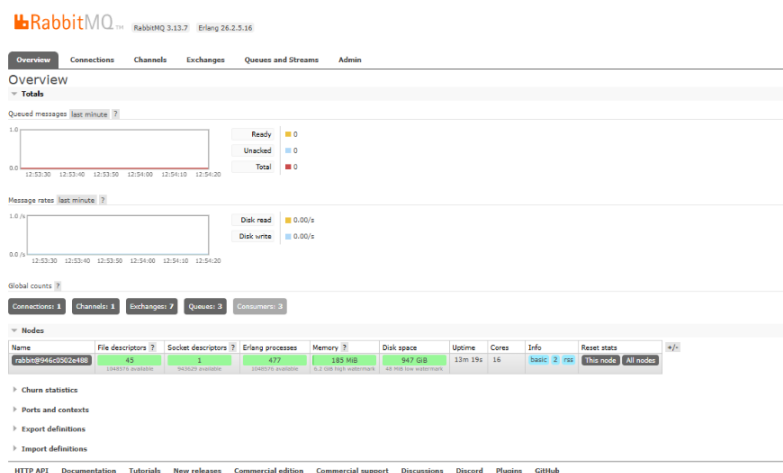
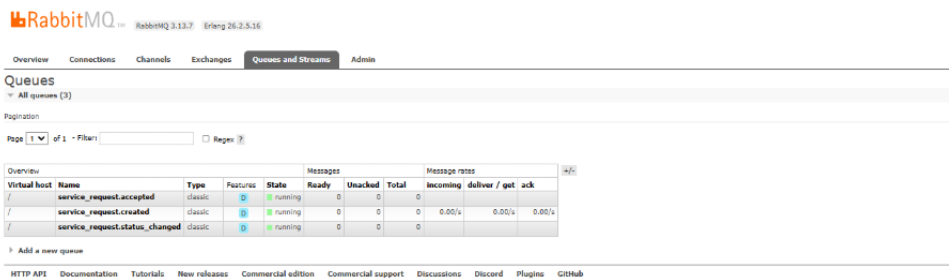


Figura 3 – Filas ativas no painel



Criação de solicitação de serviço: Ao realizar uma requisição POST /api/service-requests autenticada como cliente, o backend salvou a solicitação no banco de dados e publicou imediatamente o evento service_request.created na fila correspondente. O consumidor associado processou a mensagem de forma assíncrona, e registrou no log a notificação que seria enviada. O gráfico de mensagens no painel do RabbitMQ confirmou a entrada e o processamento da mensagem, com o contador de mensagens entregues incrementado após o ACK do consumidor. Representados abaixo pelas figuras 4 e 5:

Figura 4 – Log de criação da Solicitação

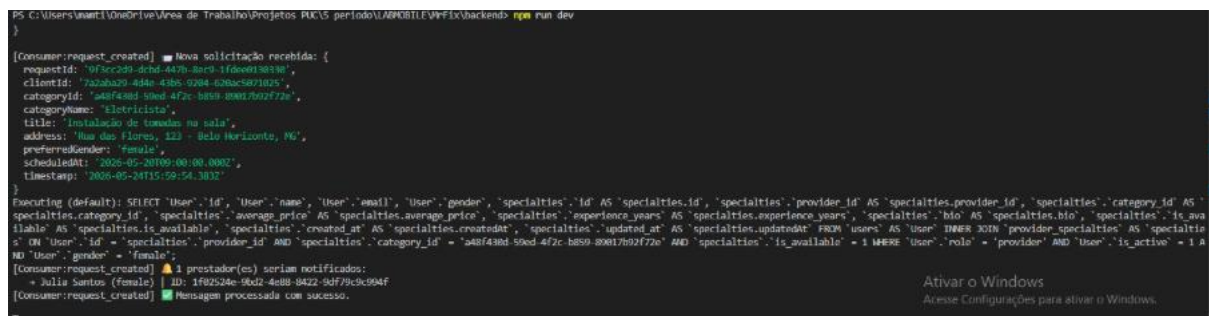
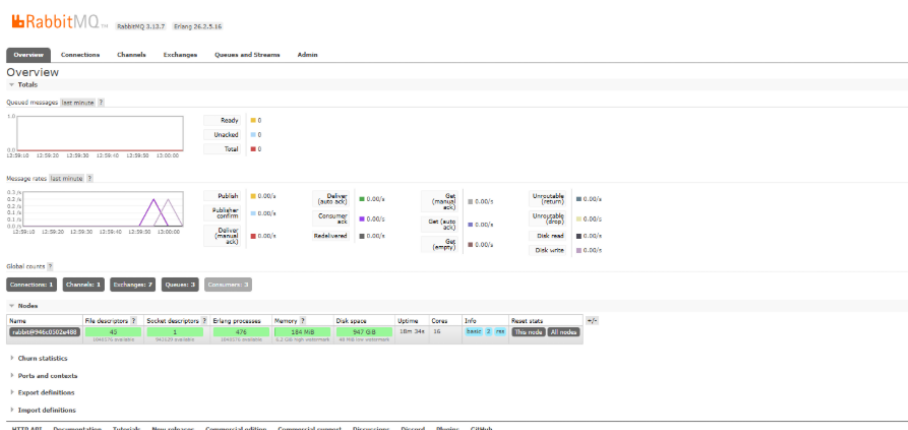


Figura 5 – Registro no painel do Rabbit



Aceite de solicitação por parte do prestador: Ao realizar uma requisição PATCH /api/service-requests/:id/status com status: accepted autenticada como prestadora, o backend publicou simultaneamente dois eventos: service_request.accepted e service_request.status_changed. Os consumidores correspondentes processaram ambas as mensagens de forma independente e assíncrona, notificando o cliente sobre o aceite e registrando a transição de status pending para accepted. O gráfico do painel RabbitMQ refletiu o pico de duas mensagens processadas em sequência, confirmando a comunicação assíncrona sem nenhuma chamada REST direta entre os componentes. Esses eventos estão representados pelas figuras 6 e 7.

Figura 6 – Log de aceite de solicitação

```
nodejs - 2026-05-24 13:05:04 - 2026-05-24 13:05:04 TERMINAL nodejs - 2026-05-24 13:05:04 - 2026-05-24 13:05:04
PS C:\Users\nanti\OneDrive\Área de Trabalho\Projetos PUC\5 periodo\LDAMD\UE\VF+lx\backend> npm run dev
timestamp: '2026-05-24T10:13:05.913Z'
}
Executing (default): SELECT `id`, `name`, `email` FROM `users` AS `User` WHERE `User`.`id` = '7124677f-8eaf-4f72-9181-4f19917b2906';
[Consumer:request_accepted] Solicitação aceita: {
  requestId: 'e8f0619f-6d79-4974-9d12-2d4808950ca9',
  clientId: '7124677f-8eaf-4f72-9181-4f19917b2906',
  providerId: '4632a853-d6ce-42e7-8f6f-2478571af1aa',
  providerName: 'Julia Santos',
  timestamp: '2026-05-24T10:13:05.917Z'
}
Executing (default): SELECT `id`, `name`, `email` FROM `users` AS `User` WHERE `User`.`id` = '7124677f-8eaf-4f72-9181-4f19917b2906';
[Consumer:status_changed] Notificando Ana Oliveira (ana@mfix.com):
  Solicitação: e8f0619f-6d79-4974-9d12-2d4808950ca9
  accepted = accepted
  Mensagem: sua solicitação foi aceita por um prestador!
[Consumer:status_changed] Mensagem processada com sucesso.
[Consumer:request_accepted] Notificando cliente Ana Oliveira:
  + Julia Santos aceitou sua solicitação (ID: e8f0619f-6d79-4974-9d12-2d4808950ca9)
[Consumer:request_accepted] Mensagem processada.
```

Figura 7 – Registro do evento no Painel

